

Group Contributions

Name	Role	Contribution
Sydonae England		
Brandon Ellington		
Jhanel Lalor		
Alexia Barrows		
Sean Spence		

Entity-Relationship Diagram (ERD)

The database has been designed with 12 normalized tables. All tables are in Third Normal Form (3NF), eliminating transitive dependencies. The ERD below describes all entities, their attributes, and relationships.

Entity Summary

Table	Primary Key	Key Attributes / Foreign Keys
users	userID	username, password, email, fname, lname, role, created_at
courses	courseID	ccode, cname, description, created_by (FK - users), lecturerID, created_at
enrollments	enrollmentID	courseID (FK - courses), studentID (FK - users), enrolled_at
calendar_events	eventID	courseID (FK - courses), title, description, event_date, start_time, end_time, created_by (FK - users)
forums	forumID	courseID (FK - courses), title, created_by (FK - users), created_at
threads	threadID	forumID (FK - forums), title, body, created_by (FK - users), created_at
replies	replyID	threadID (FK - threads), parentID (FK - replies), body, created_by (FK - users), created_at
sections	sectionID	courseID (FK - courses), title, position
items	itemID	sectionID (FK - sections), type, title, url, filepath, uploaded_by (FK - users), uploaded_at
assignments	assignmentID	courseID (FK - courses), title, description, duedate, maxgrade, created_by (FK - users), created_at
submissions	subID	assignmentID (FK - assignments), studID (FK - users), filepath, submitted_at, grade, graded_by (FK - users), graded_at

Relationships (not sure if this is right also)

- users → courses: One admin creates many courses (1:N)
- courses.lecturerID → users: One course has exactly one lecturer via lecturerID column (1:1)
- users → enrollments → courses: Many students enroll in many courses (M:N via enrollments)
- courses → calendar_events: One course has many events (1:N)
- courses → forums: One course has many forums (1:N)
- forums → threads: One forum has many threads (1:N)
- threads → replies: One thread has many replies; replies can be nested (self-referencing 1:N)
- courses → sections → items: Courses have sections, and sections have items (hierarchical 1:N)
- courses → assignments → submissions: Courses have assignments; students submit to assignments (1:N each)

Normalization Notes

1NF: All attributes contain atomic values. No repeating groups.

2NF: All non-key attributes depend on the full primary key (composite keys used only in junction tables).

3NF: No transitive dependencies. For example, student details are stored in users, not duplicated in enrollments.

Database Creation SQL Scripts

The following SQL script creates the full database schema, including all tables, foreign keys, views, and optimization indexes. This script targets MySQL.

```
-- =====
-- COMP3161 Course Management System - Database Creation Script
-- =====

CREATE DATABASE IF NOT EXISTS course_management;
USE course_management;

-- Users Table
CREATE TABLE users (
  userID INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE,
  fname VARCHAR(50) NOT NULL,
  lname VARCHAR(50) NOT NULL,
  role ENUM('admin','lecturer','student') NOT NULL DEFAULT 'student',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Courses Table
CREATE TABLE courses (
  courseID INT AUTO_INCREMENT PRIMARY KEY,
  ccode VARCHAR(20) NOT NULL UNIQUE,
  cname VARCHAR(100) NOT NULL,
  description TEXT,
  created_by INT NOT NULL,
  lecturerID INT DEFAULT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (created_by) REFERENCES users(userID),
  FOREIGN KEY (lecturerID) REFERENCES users(userID)
);

-- Course Enrollment (Students)
CREATE TABLE enrollments (
  enrollmentID INT AUTO_INCREMENT PRIMARY KEY,
  courseID INT NOT NULL,
  studID INT NOT NULL,
  enrolled_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_enrollment (courseID, studID),
  FOREIGN KEY (courseID) REFERENCES courses(courseID),
  FOREIGN KEY (studID) REFERENCES users(userID)
);

-- Calendar Events
CREATE TABLE calendar_events (
  eventID INT AUTO_INCREMENT PRIMARY KEY,
  courseID INT NOT NULL,
  title VARCHAR(150) NOT NULL,
  description TEXT,
  event_date DATE NOT NULL,
  start_time TIME,
  end_time TIME,
  created_by INT NOT NULL,
```

```

        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (courseID) REFERENCES courses(courseID),
        FOREIGN KEY (created_by) REFERENCES users(userID)
    );

-- Forums
CREATE TABLE forums (
    forumID INT AUTO_INCREMENT PRIMARY KEY,
    courseID INT NOT NULL,
    title VARCHAR(150) NOT NULL,
    created_by INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (courseID) REFERENCES courses(courseID),
    FOREIGN KEY (created_by) REFERENCES users(userID)
);

-- Discussion Threads
CREATE TABLE threads (
    threadID INT AUTO_INCREMENT PRIMARY KEY,
    forumID INT NOT NULL,
    title VARCHAR(200) NOT NULL,
    body TEXT NOT NULL,
    created_by INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (forumID) REFERENCES forums(forumID),
    FOREIGN KEY (created_by) REFERENCES users(userID)
);

-- Thread Replies (self-referencing for nested replies)
CREATE TABLE replies (
    replyID INT AUTO_INCREMENT PRIMARY KEY,
    threadID INT NOT NULL,
    parentID INT DEFAULT NULL,
    body TEXT NOT NULL,
    created_by INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (threadID) REFERENCES threads(threadID),
    FOREIGN KEY (parentID) REFERENCES replies(replyID),
    FOREIGN KEY (created_by) REFERENCES users(userID)
);

-- Course Content Sections
CREATE TABLE sections (
    sectionID INT AUTO_INCREMENT PRIMARY KEY,
    courseID INT NOT NULL,
    title VARCHAR(150) NOT NULL,
    position INT NOT NULL DEFAULT 0,
    FOREIGN KEY (courseID) REFERENCES courses(courseID)
);

-- Course Content Items
CREATE TABLE items (
    itemID INT AUTO_INCREMENT PRIMARY KEY,
    sectionID INT NOT NULL,
    type ENUM('link','file','slide') NOT NULL,
    title VARCHAR(150) NOT NULL,
    url VARCHAR(500),
    filepath VARCHAR(500),
    uploaded_by INT NOT NULL,
    uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (sectionID) REFERENCES sections(sectionID),
    FOREIGN KEY (uploaded_by) REFERENCES users(userID)
);

```

```

);

-- Assignments
CREATE TABLE assignments (
  assignmentID INT AUTO_INCREMENT PRIMARY KEY,
  courseID INT NOT NULL,
  title VARCHAR(150) NOT NULL,
  description TEXT,
  duedate DATETIME,
  maxgrade DECIMAL(5,2) NOT NULL DEFAULT 100.00,
  created_by INT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (courseID) REFERENCES courses(courseID),
  FOREIGN KEY (created_by) REFERENCES users(userID)
);

-- Assignment Submissions
CREATE TABLE submissions (
  subID INT AUTO_INCREMENT PRIMARY KEY,
  assignmentID INT NOT NULL,
  studID INT NOT NULL,
  filepath VARCHAR(500),
  submitted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  grade DECIMAL(5,2) DEFAULT NULL,
  graded_by INT DEFAULT NULL,
  graded_at TIMESTAMP DEFAULT NULL,
  UNIQUE KEY uq_submission (assignmentID, studID),
  FOREIGN KEY (assignmentID) REFERENCES assignments(assignmentID),
  FOREIGN KEY (studID) REFERENCES users(userID),
  FOREIGN KEY (graded_by) REFERENCES users(userID)
);

-- =====
-- VIEWS
-- =====

-- Courses with 50 or more students
CREATE VIEW vw_courses_50_plus AS
SELECT c.courseID, c.ccode, c.cname, COUNT(e.studID) AS student_count
FROM courses c
JOIN enrollments e ON c.courseID = e.courseID
GROUP BY c.courseID, c.ccode, c.cname
HAVING COUNT(e.studID) >= 50;

-- Students enrolled in 5 or more courses
CREATE VIEW vw_students_5_plus_courses AS
SELECT u.userID, u.username, u.fname, u.lname, COUNT(e.courseID) AS course_count
FROM users u
JOIN enrollments e ON u.userID = e.studID
WHERE u.role = 'student'
GROUP BY u.userID, u.username, u.fname, u.lname
HAVING COUNT(e.courseID) >= 5;

-- Lecturers teaching 3 or more courses
CREATE VIEW vw_lecturers_3_plus_courses AS
SELECT u.userID, u.username, u.fname, u.lname, COUNT(c.courseID) AS course_count
FROM users u
JOIN courses c ON u.userID = c.lecturerID
WHERE u.role = 'lecturer'
GROUP BY u.userID, u.username, u.fname, u.lname
HAVING COUNT(c.courseID) >= 3;

```

```
-- Top 10 most enrolled courses
CREATE VIEW vw_top_10_enrolled_courses AS
SELECT c.courseID, c.ccode, c.cname, COUNT(e.studID) AS student_count
FROM courses c
JOIN enrollments e ON c.courseID = e.courseID
GROUP BY c.courseID, c.ccode, c.cname
ORDER BY student_count DESC
LIMIT 10;
```

```
-- Top 10 students by overall average
CREATE VIEW vw_top_10_students_avg AS
SELECT u.userID, u.username, u.fname, u.lname,
       ROUND(AVG(s.grade), 2) AS overall_average
FROM users u
JOIN submissions s ON u.userID = s.studID
WHERE s.grade IS NOT NULL AND u.role = 'student'
GROUP BY u.userID, u.username, u.fname, u.lname
ORDER BY overall_average DESC
LIMIT 10;
```

```
-- =====
-- INDEXES FOR OPTIMIZATION
-- =====

CREATE INDEX idx_enrollments_student ON enrollments(studID);
CREATE INDEX idx_enrollments_course ON enrollments(courseID);
CREATE INDEX idx_submissions_student ON submissions(studID);
CREATE INDEX idx_calendar_date      ON calendar_events(event_date);
CREATE INDEX idx_threads_forum      ON threads(forumID);
CREATE INDEX idx_replies_thread     ON replies(threadID);
CREATE INDEX idx_replies_parent     ON replies(parentID);
CREATE INDEX idx_content_section    ON items(sectionID);
CREATE INDEX idx_courses_lecturer   ON courses(lecturerID);
```


Data Population SQL Scripts

The following scripts insert the required data: 1 admin, 200 lecturers, 100,000 students, 200 courses, enrollments (3–6 per student), calendar events, forums, threads, and assignments. Stored procedures are used for bulk insertion.

```
-- =====
-- COMP3161 - Data Population Script
-- =====

-- 1. Insert Admin
INSERT INTO users (username, password, email, fname, lname, role) VALUES
('admin', '$2b$12$adminHashedPassword', 'admin@school.edu', 'System', 'Admin', 'admin');

-- 2. Insert Lecturers (200 lecturers to cover 200 courses, max 5 each)
INSERT INTO users (username, password, email, fname, lname, role)
SELECT
  CONCAT('lecturer', n),
  '$2b$12$m7a3gbvLiRMOXtbspJ1.v.HFbgOch5LRo3QV7siwcRRG0IND69vdi',
  CONCAT('lecturer', n, '@school.edu'),
  ELT(1 + MOD(n, 60),
    'Akeem','Shanelle','Marlon','Latoya','Damion','Patrice','Rohan','Nadine','Delroy','Tashana',
    'Fabian','Shenique','Winston','Beverley','Clive','Kezia','Orville','Simone','Dwayne','Camille',
    'Jermaine','Tenesha','Leroy','Monique','Andre','Dionne','Carlton','Stacey','Everton','Keturah',
    'Fitzroy','Paulette','Garfield','Novelette','Courtney','Hyacinth','Donovan','Annmarie','Lancelot','Sonia',
    'Oliver','Emma','William','Charlotte','George','Amelia','Edward','Victoria','James','Florence',
    'Carlos','Isabella','Miguel','Valentina','Jean','Marie','Pierre','Sophie','Antoine','Isabelle'),
  ELT(1 + MOD(n * 3, 70),
    'Campbell','Morrison','Brown','Francis','Henry','Gordon','Reid','Grant','Thomas','Barrett',
    'Bailey','Blake','Clarke','Douglas','Edwards','Fletcher','Gibson','Hamilton','Irving','James',
    'Beckford','Chin','Wong','Chang','Lee','Burke','Stewart','McLean','McIntosh','McNab',
    'Levy','Lyn','Hylton','Scarlett','Samuels','Ricketts','Daley','Facey','Lawson','Pinnock',
    'Smith','Jones','Davies','Evans','Wilson','Taylor','Hughes','Roberts','Walker','White',
    'Garcia','Rodriguez','Fernandez','Lopez','Gonzalez','Ramirez','Sanchez','Perez','Torres','Flores',
    'Bernard','Dubois','Laurent','Martin','Moreau','Rousseau','Fontaine','Leclerc','Dupont','Garnier'),
  'lecturer'
FROM (
  SELECT a.N + b.N * 10 + c.N * 100 + 1 AS n
  FROM
    (SELECT 0 AS N UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4
     UNION SELECT 5 UNION SELECT 6 UNION SELECT 7 UNION SELECT 8 UNION SELECT 9) a,
    (SELECT 0 AS N UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4
     UNION SELECT 5 UNION SELECT 6 UNION SELECT 7 UNION SELECT 8 UNION SELECT 9) b,
    (SELECT 0 AS N UNION SELECT 1) c
) nums WHERE n <= 200;

-- 3. Insert 100,000 Students using stored procedure
DELIMITER $$
CREATE PROCEDURE insert_students()
BEGIN
  DECLARE i INT DEFAULT 1;
  WHILE i <= 100000 DO
    INSERT INTO users (username, password, email, fname, lname, role)
    VALUES (
      CONCAT('student', i),
      '$2b$12$m7a3gbvLiRMOXtbspJ1.v.HFbgOch5LRo3QV7siwcRRG0IND69vdi',
      CONCAT('student', i, '@school.edu'),
      ELT(1 + MOD(i, 100),
        'Jaheim','Tavia','Omari','Shanique','Kevon','Brianna','Romario','Destiny','Tajay','Kadia',
```

```

'Dejour','Jade','Tyrese','Jada','Jovan','Danielle','Rasheem','Gabrielle','Malik','Imani',
'Kymani','Rena','Jahvon','Kiara','Devante','Shaneka','Lemar','Chevanese','Tariq','Jasmine',
'Omarion','Tameka','Zidane','Nadia','Kemar','Aaliyah','Kadeem','Staci','Deshawn','Petrina',
'Javonte','Ashanti','Tremaine','Rochelle','Kimani','Oneika','Tyrell','Sasha','Mikael','Shelly',
'Jaheem','Tanesha','Romairo','Jhanae','Tavaris','Kellisha','Devonte','Tamara','Rashaun','Kayla',
'Omari','Zanelle','Kaylia','Tricia','Shanice','Tiffany','Zanaya','Aaliya','Zara','Tiana',
'Javion','Briella','Romaine','Gabriella','Dejean','Daniella','Tareke','Imari','Kavon','Janae',
'Liam','Chloe','Noah','Zoe','Ethan','Mia','Lucas','Ella','Mason','Ava',
'Diego','Isabella','Mateo','Sofia','Carlos','Valentina','Miguel','Camila','Rafael','Lucia'),
ELT(1 + MOD(i * 7, 90),
'Campbell','Morrison','Brown','Francis','Henry','Gordon','Reid','Grant','Thomas','Barrett',
'Bailey','Blake','Clarke','Douglas','Edwards','Fletcher','Gibson','Hamilton','Irving','James',
'Beckford','Chin','Wong','Chang','Lee','Burke','Stewart','McLean','McIntosh','McNab',
'Levy','Lyn','Hylton','Scarlett','Samuels','Ricketts','Daley','Facey','Lawson','Pinnock',
'Bennett','Senior','Anderson','Williams','Johnson','Davis','Jackson','White','Harris','Martin',
'Marley','Tosh','Ranglin','Dekker','Perry','Kong','Livingston','Jobson','Wilmot','Palmer',
'Smith','Jones','Davies','Evans','Wilson','Taylor','Hughes','Roberts','Walker','Allen',
'Garcia','Rodriguez','Fernandez','Lopez','Gonzalez','Ramirez','Sanchez','Perez','Torres','Flores',
'Bernard','Dubois','Laurent','Moreau','Rousseau','Fontaine','Leclerc','Dupont','Garnier','Simon'),
'student'
);
SET i = i + 1;
END WHILE;
END$$
DELIMITER ;
CALL insert_students();
DROP PROCEDURE insert_students;

```

-- 4. Insert 200 Courses

```

INSERT INTO courses (ccode, cname, description, created_by, lecturerID)
SELECT
  CONCAT('CRS', LPAD(n, 3, '0')),
  CONCAT(
    ELT(1 + MOD(n, 8),
      'Introduction to ', 'Advanced ', 'Principles of ', 'Foundations of ',
      'Applied ', 'Theory of ', 'Topics in ', 'Studies in '),
    ELT(1 + MOD(n, 12),
      'Computer Science', 'Mathematics', 'Physics', 'Chemistry',
      'Biology', 'Engineering', 'Data Science', 'Networks',
      'Databases', 'Algorithms', 'Software Engineering', 'Cybersecurity')
  ),
  CONCAT('Core course number ', n),
  1,
  2 + MOD(n - 1, 200) --lecturerID: cycles through lecturers (userID 2 - 201)
FROM (
  SELECT a.N + b.N * 10 + c.N * 100 + 1 AS n
  FROM
    (SELECT 0 AS N UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4
     UNION SELECT 5 UNION SELECT 6 UNION SELECT 7 UNION SELECT 8 UNION SELECT 9) a,
    (SELECT 0 AS N UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4
     UNION SELECT 5 UNION SELECT 6 UNION SELECT 7 UNION SELECT 8 UNION SELECT 9) b,
    (SELECT 0 AS N UNION SELECT 1) c
) nums WHERE n <= 200;

```

-- 5. Redistribute lecturers so some teach 3+ courses (max 5)

```

UPDATE courses
SET lecturerID = 2 + MOD(courseID, 67)
WHERE courseID > 0;

```

-- 6. Enroll Students (3-6 courses per student) using stored procedure

```
DELIMITER $$
CREATE PROCEDURE enroll_students()
BEGIN
    DECLARE sid INT;
    DECLARE min_userID INT;
    DECLARE course_count INT;
    DECLARE cid INT;
    DECLARE enrolled INT;

    SET min_userID = 202; -- students start after admin + 200 lecturers

    SET sid = min_userID;
    WHILE sid <= min_userID + 99999 DO
        SET enrolled = 0;
        SET course_count = 3 + MOD(sid, 4); -- 3 to 6 courses

        WHILE enrolled < course_count DO
            SET cid = 1 + MOD(sid * 7 + enrolled * 13, 200);
            INSERT IGNORE INTO enrollments (courseID, studID)
            VALUES (cid, sid);
            SET enrolled = enrolled + 1;
        END WHILE;

        SET sid = sid + 1;
    END WHILE;
END$$
DELIMITER ;
CALL enroll_students();
DROP PROCEDURE enroll_students;
```

-- 7. Insert Calendar Events (2 per course)

```
INSERT INTO calendar_events (courseID, title, description, event_date, start_time, end_time, created_by)
SELECT
    c.courseID,
    CONCAT('Lecture - Week ', MOD(c.courseID, 14) + 1),
    'Regular scheduled lecture',
    DATE_ADD('2025-01-13', INTERVAL MOD(c.courseID, 90) DAY),
    '09:00:00', '11:00:00',
    1
FROM courses c
UNION ALL
SELECT
    c.courseID,
    CONCAT('Assignment Due - ', c.ccode),
    'Assignment submission deadline',
    DATE_ADD('2025-02-01', INTERVAL MOD(c.courseID, 60) DAY),
    '23:59:00', '23:59:00',
    1
FROM courses c;
```

-- 8. Insert Forums (1 per course)

```
INSERT INTO forums (courseID, title, created_by)
SELECT courseID, CONCAT('General Discussion - ', ccode), 1
FROM courses;
```

-- 9. Insert Discussion Threads (2 per forum)

```
INSERT INTO threads (forumID, title, body, created_by)
```

```

SELECT f.forumID,
    'Welcome to the course!',
    'Please introduce yourself and share your expectations for this course.',
    1
FROM forums f
UNION ALL
SELECT f.forumID,
    'Questions and Answers',
    'Use this thread to ask any course-related questions.',
    1
FROM forums f;

```

–10. Insert Sections (2 sections per course)

```

INSERT INTO sections (courseID, title, position)
SELECT courseID, CONCAT('Week 1 - ', ccode), 1 FROM courses
UNION ALL
SELECT courseID, CONCAT('Week 2 - ', ccode), 2 FROM courses;

```

–11. Insert items (2 items per section (links))

```

INSERT INTO items (sectionID, type, title, url, uploaded_by)
SELECT s.sectionID, 'link', CONCAT('Lecture Notes - ', s.title),
    CONCAT('https://example.com/notes/', s.sectionID), 1
FROM sections s
UNION ALL
SELECT s.sectionID, 'link', CONCAT('Slides - ', s.title),
    CONCAT('https://example.com/slides/', s.sectionID), 1
FROM sections s;

```

-- 12. Insert Assignments (2 per course)

```

INSERT INTO assignments (courseID, title, description, dueDate, maxGrade, created_by)
SELECT
    c.courseID,
    CONCAT('Assignment 1 - ', c.ccode),
    'First assignment for this course.',
    DATE_ADD('2025-02-15', INTERVAL MOD(c.courseID, 30) DAY),
    100.00,
    1
FROM courses c
UNION ALL
SELECT
    c.courseID,
    CONCAT('Assignment 2 - ', c.ccode),
    'Second assignment for this course.',
    DATE_ADD('2025-03-15', INTERVAL MOD(c.courseID, 30) DAY),
    100.00,
    1
FROM courses c;

```

REST API — Flask Implementation

The API was built with Python Flask and uses raw MySQL queries (no ORM). JWT authentication was also implemented via flask-jwt-extended. The server runs on port 5000.

Setup & Installation

- Python 3.10+ required
- Install dependencies: `pip install flask flask-jwt-extended mysql-connector-python bcrypt`
- Set environment variables: `DB_HOST`, `DB_USER`, `DB_PASS`, `JWT_SECRET`
- Run with: `python app.py`

API Endpoints Reference

Method	Endpoint	Description	Access
POST	/api/register	Register a new user	Public
POST	/api/login	Login and receive JWT token	Public
POST	/api/courses	Create a new course	Admin
GET	/api/courses	Retrieve all courses	Auth
GET	/api/courses/student/{studID}	Courses for a specific student	Auth
GET	/api/courses/lecturer/{lecturerID}	Courses taught by a lecturer	Auth
POST	/api/courses/{courseID}/enroll	Student enrolls in a course	Auth
POST	/api/courses/{courseID}/assign-lecturer	Assign lecturer to a course	Admin
GET	/api/courses/{courseID}/members	Retrieve course members	Auth
GET	/api/courses/{courseID}/events	Get all events for a course	Auth
POST	/api/courses/{courseID}/events	Create a calendar event	Lecturer/Admin
GET	/api/students/{studID}/events?date=YYYY-MM-DD	Get student events for a date	Auth
GET	/api/courses/{courseID}/forums	Get all forums for a course	Auth
POST	/api/courses/{courseID}/forums	Create a forum	Auth
GET	/api/forums/{forumID}/threads	Get all threads in a forum	Auth
POST	/api/forums/{forumID}/threads	Create a new discussion thread	Auth

Method	Endpoint	Description	Access
POST	/api/threads/{threadID}/replies	Reply to a thread (nested supported)	Auth
GET	/api/courses/{courseID}/content	Retrieve course content by section	Auth
POST	/api/sections/{sectionID}/content	Add content item to a section	Lecturer/Admin
POST	/api/courses/{courseID}/assignments/{id}/submit	Submit an assignment	Student
POST	/api/submissions/{subID}/grade	Grade a submission	Lecturer/Admin
GET	/api/reports/courses-50-plus	Courses with 50+ students	Admin
GET	/api/reports/students-5-plus-courses	Students in 5+ courses	Admin
GET	/api/reports/lecturers-3-plus-courses	Lecturers teaching 3+ courses	Admin
GET	/api/reports/top-10-enrolled	Top 10 most enrolled courses	Auth
GET	/api/reports/top-10-students	Top 10 students by average grade	Admin/Lecturer

Full Flask Application Code

```
from flask import Flask, request, make_response, jsonify

from flask_jwt_extended import JWTManager, create_access_token, jwt_required, get_jwt, get_jwt_identity

from flask_httpauth import HTTPBasicAuth

import mysql.connector

import bcrypt

from functools import wraps

from dotenv import load_dotenv

import os


app = Flask(__name__)


auth = HTTPBasicAuth()

load_dotenv()

app.config['JWT_SECRET_KEY'] = os.getenv('JWT_SECRET_KEY')

jwt = JWTManager(app)


def get_db_connection():

    connection = mysql.connector.connect(

        user=os.getenv('DB_USER'),

        password=os.getenv('DB_PASSWORD'),

        database=os.getenv('DB_NAME'),

        host=os.getenv('DB_HOST')

    )

    return connection
```

```

def role_required(*roles):

    def wrapper(fn):

        @wraps(fn)

        @jwt_required()

        def decorated(*args, **kwargs):

            role = get_jwt()['role']

            if role not in roles:

                return jsonify({"message": "Access denied"}), 403

            return fn(*args, **kwargs)

        return decorated

    return wrapper


# GET ALL USERS

@app.route('/users', methods=['GET'])

def get_users():

    try:

        with get_db_connection() as cnx:

            with cnx.cursor() as cursor:

                cursor.execute("SELECT userID, username, email, fname, lname, role FROM users")

                rows = cursor.fetchall()

                users = [{"userID": r[0], "username": r[1], "email": r[2], "fname": r[3], "lname": r[4], "role": r[5]} for r in
rows]

                return jsonify(users), 200

            except Exception as e:

                return jsonify({"message": str(e)}), 400


# REGISTER USER

```



```
@app.route('/register', methods=['POST'])
```

```
def register():
```

```
    try:
```

```
        new_user = request.get_json()
```

```
        username = new_user['username']
```

```
        password = new_user['password']
```

```
        email = new_user['email']
```

```
        fname = new_user['fname']
```

```
        lname = new_user['lname']
```

```
        role = new_user['role']
```

```
    with get_db_connection() as cnx:
```

```
        with cnx.cursor() as cursor:
```

```
            cursor.execute("SELECT * FROM users WHERE username = %s", (username,))
```

```
            if cursor.fetchone():
```

```
                return jsonify({"message": "User already exists"}), 400
```

```
            hashed = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt()).decode('utf-8')
```

```
            cursor.execute("INSERT INTO users (username, password, email, fname, lname, role) VALUES (%s, %s, %s, %s, %s, %s)",
```

```
                            (username, hashed, email, fname, lname, role))
```

```
            cnx.commit()
```

```
        return jsonify({"message": "Account created successfully"}), 201
```

```
    except Exception as e:
```

```
        return jsonify({"message": str(e)}), 400
```

```
# USER LOGIN
```

```

@app.route('/login', methods=['POST'])

def login():

    try:

        username = request.json.get('username', None)

        password = request.json.get('password', None)


        with get_db_connection() as cnx:

            with cnx.cursor() as cursor:

                cursor.execute("SELECT userID, username, password, fname, lname, role FROM users WHERE username =
%s", (username,))

                row = cursor.fetchone()

                if row is None:

                    return make_response({'error': 'Invalid credentials'}, 401)

                userID, db_username, stored_hash, fname, lname, role = row

                if bcrypt.checkpw(password.encode('utf-8'), stored_hash.encode('utf-8')):

                    access_token = create_access_token(identity=str(userID), additional_claims={"role": role})

                    return make_response({'success': 'Login successful', 'access_token': access_token}, 200)

                else:

                    return make_response({'error': 'Invalid credentials'}, 401)

    except Exception as e:

        return jsonify({"message": str(e)}), 400


# CREATE COURSE

@app.route('/courses', methods=['POST'])

@role_required('admin')

def create_course():

    try:

```

```

new_course = request.get_json()

ccode = new_course['ccode']

cname = new_course['cname']

desc = new_course['description']

created_by = get_jwt_identity()

with get_db_connection() as cnx:

    with cnx.cursor() as cursor:

        cursor.execute("SELECT * FROM courses WHERE ccode = %s", (ccode,))

        if cursor.fetchone():

            return jsonify({"message": "Course already exists"}), 400

        cursor.execute("INSERT INTO courses (ccode, cname, description, created_by) VALUES (%s, %s, %s,
%s)",

                        (ccode, cname, desc, created_by))

        cnx.commit()

    return jsonify({"message": "Course created successfully"}), 201

except Exception as e:

    return jsonify({"message": str(e)}), 400


# RETRIEVE ALL COURSES

@app.route('/courses', methods=['GET'])

def get_courses():

    try:

        with get_db_connection() as cnx:

            with cnx.cursor() as cursor:

                cursor.execute("SELECT courseID, ccode, cname, description FROM courses")

                rows = cursor.fetchall()

                courses = [{"courseID": r[0], "ccode": r[1], "cname": r[2], "description": r[3]} for r in rows]

```

```

    return jsonify(courses), 200

except Exception as e:

    return jsonify({"message": str(e)}), 400

# RETRIEVE COURSES BY STUDENT

@app.route('/courses/student/<int:studID>', methods=['GET'])

def get_course_by_student(studID):

    try:

        with get_db_connection() as cnx:

            with cnx.cursor() as cursor:

                cursor.execute("""

                    SELECT c.ccode, c.cname, c.description

                    FROM courses c JOIN enrollments e ON c.courseID = e.courseID

                    WHERE e.studID = %s""", (studID,))

                rows = cursor.fetchall()

                courses = [{"ccode": r[0], "cname": r[1], "description": r[2]} for r in rows]

            return jsonify(courses), 200

    except Exception as e:

        return jsonify({"message": str(e)}), 400

```

```

# RETRIEVE COURSES BY LECTURER

@app.route('/courses/lecturer/<int:lecturerID>', methods=['GET'])

def get_course_by_lecturer(lecturerID):

    try:

        with get_db_connection() as cnx:

            with cnx.cursor() as cursor:

                cursor.execute("""

```

```

        SELECT c.ccode, c.cname, c.description, u.fname, u.lname
        FROM courses c JOIN users u ON c.lecturerID = u.userID
        WHERE c.lecturerID = %s""", (lecturerID,))

    rows = cursor.fetchall()

    courses = [{"ccode": r[0], "cname": r[1], "description": r[2], "fname": r[3], "lname": r[4]} for r in rows]

    return jsonify(courses), 200

except Exception as e:

    return jsonify({"message": str(e)}), 400

# REGISTER FOR COURSE

@app.route('/courses/<int:courseID>/enroll', methods=['POST'])
@jwt_required()
def enroll_student(courseID):

    try:

        data = request.get_json()

        studID = data ['studID']

        with get_db_connection() as cnx:

            with cnx.cursor() as cursor:

                cursor.execute("INSERT INTO enrollments (courseID, studID) VALUES (%s, %s)", (courseID, studID))

                cnx.commit()

            return jsonify({"message": "Student enrolled successfully"}), 201

    except Exception as e:

        return jsonify({"message": str(e)}), 400

#ASSIGN LECTURER

@app.route('/courses/<int:courseID>/assign-lecturer', methods=['POST'])
@role_required('admin')

```

```

def assign_lecturer(courseID):

    try:

        data = request.get_json()

        lecturerID = data['lecturerID']

        with get_db_connection() as cnx:

            with cnx.cursor() as cursor:

                cursor.execute("UPDATE courses SET lecturerID = %s WHERE courseID = %s", (lecturerID, courseID))

                cnx.commit()

            return jsonify({"message": "Lecturer assigned successfully"}), 200

    except Exception as e:

        return jsonify({"message": str(e)}), 400

```

RETRIEVE MEMBERS

```
@app.route('/courses/<int:courseID>/members', methods=['GET'])
```

```
@jwt_required()
```

```

def retrieve_members(courseID):

    try:

        with get_db_connection() as cnx:

            with cnx.cursor() as cursor:

                cursor.execute("""

                    SELECT u.userID, u.username, u.fname, u.lname, u.role

                    FROM users u JOIN enrollments e ON u.userID = e.studID

                    WHERE e.courseID = %s""", (courseID,))

                rows = cursor.fetchall()

                members = [{"userID": r[0], "username": r[1], "fname": r[2], "lname": r[3], "role": r[4]} for r in rows]

            return jsonify(members), 200

    except Exception as e:

```

```
return jsonify({"message": str(e)}), 400
```

```
# RETRIEVE CALENDAR EVENTS
```

```
@app.route('/courses/<int:courseID>/events', methods=['GET'])
```

```
@jwt_required()
```

```
def get_events(courseID):
```

```
    try:
```

```
        with get_db_connection() as cnx:
```

```
            with cnx.cursor() as cursor:
```

```
                cursor.execute("""
```

```
                    SELECT eventID, title, description, event_date, start_time, end_time
```

```
                    FROM calendar_events WHERE courseID = %s""", (courseID,))
```

```
                rows = cursor.fetchall()
```

```
                events = [{"eventID": r[0], "title": r[1], "description": r[2], "event_date": str(r[3]), "start_time": str(r[4]),  
"end_time": str(r[5])} for r in rows]
```

```
                return jsonify(events), 200
```

```
            except Exception as e:
```

```
                return jsonify({"message": str(e)}), 400
```

```
# GET EVENT BY DATE
```

```
@app.route('/students/<int:studID>/events', methods=['GET'])
```

```
@jwt_required()
```

```
def get_event_by_date(studID):
```

```
    try:
```

```
        date = request.args.get('date')
```

```
        with get_db_connection() as cnx:
```

```
            with cnx.cursor() as cursor:
```

```

        cursor.execute("""

        SELECT ce.eventID, ce.title, ce.description, ce.event_date, ce.start_time, ce.end_time

        FROM calendar_events ce

        JOIN enrollments e ON ce.courseID = e.courseID

        WHERE e.studID = %s AND ce.event_date = %s""", (studID, date))

    rows = cursor.fetchall()

    events = [{"eventID": r[0], "title": r[1], "description": r[2], "event_date": str(r[3]), "start_time": str(r[4]),
"end_time": str(r[5])} for r in rows]

    return jsonify(events), 200

except Exception as e:

    return jsonify({"message": str(e)}), 400


# CREATE CALENDAR EVENTS

@app.route('/courses/<int:courseID>/events', methods=['POST'])
@role_required('lecturer', 'admin')
def create_event(courseID):

    try:

        data = request.get_json()

        title = data['title']

        description = data.get('description', "")

        event_date = data['event_date']

        start_time = data.get('start_time')

        end_time = data.get('end_time')

        created_by = get_jwt_identity()

        with get_db_connection() as cnx:

            with cnx.cursor() as cursor:

                cursor.execute("""

```



```

        INSERT INTO calendar_events (courseID, title, description, event_date, start_time, end_time, created_by)
        VALUES (%s, %s, %s, %s, %s, %s, %s)""" , (courseID, title, description, event_date, start_time, end_time,
        created_by))

        cnx.commit()

        return jsonify({"message": "Event created successfully"}), 201

    except Exception as e:

        return jsonify({"message": str(e)}), 400

```

#FORUMS

```

@app.route('/courses/<int:courseID>/forums', methods=['GET'])
@jwt_required()
def get_forums_by_course(courseID):

    try:

        with get_db_connection() as cnx:

            with cnx.cursor() as cursor:

                cursor.execute("SELECT forumID, title, created_at FROM forums WHERE courseID = %s", (courseID,))

                rows = cursor.fetchall()

                forums = [{"forumID": r[0], "title": r[1], "created_at": str(r[2])} for r in rows]

            return jsonify(forums), 200

    except Exception as e:

        return jsonify({"message": str(e)}), 400

```

#CREATE FORUM

```

@app.route('/courses/<int:courseID>/forums', methods=['POST'])
@jwt_required()
def create_forum(courseID):

    try:

```

```

data = request.get_json()

title = data['title']

created_by = get_jwt_identity()

with get_db_connection() as cnx:

    with cnx.cursor() as cursor:

        cursor.execute("INSERT INTO forums (courseID, title, created_by) VALUES (%s, %s, %s)", (courseID,
title, created_by))

        cnx.commit()

    return jsonify({"message": "Forum created successfully"}), 201

except Exception as e:

    return jsonify({"message": str(e)}), 400


# DISCUSSION THREADS

@app.route('/forums/<int:forumID>/threads', methods=['GET'])

@jwt_required()

def get_threads_by_forum(forumID):

    try:

        with get_db_connection() as cnx:

            with cnx.cursor() as cursor:

                cursor.execute("SELECT threadID, title, body, created_at FROM threads WHERE forumID = %s",
(forumID,))

                rows = cursor.fetchall()

                threads = [{"threadID": r[0], "title": r[1], "body": r[2], "created_at": str(r[3])} for r in rows]

            return jsonify(threads), 200

        except Exception as e:

            return jsonify({"message": str(e)}), 400


#CREATE THREAD

```

```

@app.route('/forums/<int:forumID>/threads', methods=['POST'])
@jwt_required()
def create_thread(forumID):
    try:
        data = request.get_json()
        title = data['title']
        body = data['body']
        created_by = get_jwt_identity()
        with get_db_connection() as cnx:
            with cnx.cursor() as cursor:
                cursor.execute("INSERT INTO threads (forumID, title, body, created_by) VALUES (%s, %s, %s, %s)",
                               (forumID, title, body, created_by))
                cnx.commit()
            return jsonify({"message": "Thread created successfully"}), 201
    except Exception as e:
        return jsonify({"message": str(e)}), 400

```

#THREAD REPLIES

```

@app.route('/threads/<int:threadID>/replies', methods=['POST'])
@jwt_required()
def reply_to_thread(threadID):
    try:
        data = request.get_json()
        body = data['body']
        parentID = data.get('parentID', None) # None if it's a top-level reply
        created_by = get_jwt_identity()

```

```
with get_db_connection() as cnx:
```

```
    with cnx.cursor() as cursor:
```

```
        cursor.execute("""
```

```
            INSERT INTO replies (threadID, parentID, body, created_by)
```

```
            VALUES (%s, %s, %s, %s)""",
```

```
            (threadID, parentID, body, created_by))
```

```
        cnx.commit()
```

```
    return jsonify({"message": "Reply added successfully"}), 201
```

```
except Exception as e:
```

```
    return jsonify({"message": str(e)}), 400
```

```
# COURSE CONTENT
```

```
@app.route('/courses/<int:courseID>/content', methods=['GET'])
```

```
@jwt_required()
```

```
def get_content_by_section(courseID):
```

```
    try:
```

```
        with get_db_connection() as cnx:
```

```
            with cnx.cursor() as cursor:
```

```
                cursor.execute("""
```

```
                    SELECT s.sectionID, s.title, s.position,
```

```
                           i.itemID, i.type, i.title, i.url, i.filepath
```

```
                    FROM sections s
```

```
                    LEFT JOIN items i ON s.sectionID = i.sectionID
```

```
                    WHERE s.courseID = %s
```

```
                    ORDER BY s.position, i.itemID""", (courseID,))
```

```
                rows = cursor.fetchall()
```

```

sections = {}

for r in rows:

    sid = r[0]

    if sid not in sections:

        sections[sid] = {

            "sectionID": r[0],

            "title": r[1],

            "position": r[2],

            "items": []

        }

    if r[3]: # only add item if it exists

        sections[sid]["items"].append({

            "itemID": r[3],

            "type": r[4],

            "title": r[5],

            "url": r[6],

            "filepath": r[7]

        })

```

```

return jsonify(list(sections.values())), 200

```

```

except Exception as e:

```

```

    return jsonify({"message": str(e)}), 400

```

```

#ADD CONTENT

```

```

@app.route('/sections/<int:sectionID>/content', methods=['POST'])

```

```

@role_required('lecturer', 'admin')

```

```

def add_content(sectionID):

```

```

try:

    data = request.get_json()

    type = data['type']      # 'link', 'file', or 'slide'

    title = data['title']

    url = data.get('url', None)

    filepath = data.get('filepath', None)

    uploaded_by = get_jwt_identity()


with get_db_connection() as cnx:

    with cnx.cursor() as cursor:

        cursor.execute("""

            INSERT INTO items (sectionID, type, title, url, filepath, uploaded_by)

            VALUES (%s, %s, %s, %s, %s, %s)""",

            (sectionID, type, title, url, filepath, uploaded_by))

        cnx.commit()

    return jsonify({"message": "Content added successfully"}), 201

except Exception as e:

    return jsonify({"message": str(e)}), 400

```

SUBMIT ASSIGNMENTS

```
@app.route('/courses/<int:courseID>/assignments/<int:assignmentID>/submit', methods=['POST'])
```

```
@role_required('student')
```

```
def submit_assignment(courseID, assignmentID):
```

```

try:

    data = request.get_json()

    filepath = data.get('filepath', None)

    studID = data['studID']

```

```
with get_db_connection() as cnx:
```

```
    with cnx.cursor() as cursor:
```

```
        # Check if student is enrolled in the course
```

```
        cursor.execute("""
```

```
            SELECT * FROM enrollments
```

```
            WHERE courseID = %s AND studID = %s""", (courseID, studID))
```

```
    if not cursor.fetchone():
```

```
        return jsonify({"message": "Student is not enrolled in this course"}), 403
```

```
        # Check if already submitted
```

```
        cursor.execute("""
```

```
            SELECT * FROM submissions
```

```
            WHERE assignmentID = %s AND studID = %s""", (assignmentID, studID))
```

```
    if cursor.fetchone():
```

```
        return jsonify({"message": "Assignment already submitted"}), 400
```

```
        cursor.execute("""
```

```
            INSERT INTO submissions (assignmentID, studID, filepath)
```

```
            VALUES (%s, %s, %s)""", (assignmentID, studID, filepath))
```

```
        cnx.commit()
```

```
    return jsonify({"message": "Assignment submitted successfully"}), 201
```

```
except Exception as e:
```

```
    return jsonify({"message": str(e)}), 400
```

```
#GRADE ASSIGNMENT
```

```

@app.route('/submissions/<int:subID>/grade', methods=['POST'])
@role_required('lecturer', 'admin')
def grade_assignment(subID):
    try:
        data = request.get_json()
        grade = data['grade']
        graded_by = get_jwt_identity()

        with get_db_connection() as cnx:
            with cnx.cursor() as cursor:
                # Check if submission exists
                cursor.execute("""
                    SELECT * FROM submissions
                    WHERE subID = %s""", (subID,))

                if not cursor.fetchone():
                    return jsonify({"message": "Submission not found"}), 404

                cursor.execute("""
                    UPDATE submissions
                    SET grade = %s, graded_by = %s, graded_at = NOW()
                    WHERE subID = %s""", (grade, graded_by, subID))
                cnx.commit()

            return jsonify({"message": "Submission graded successfully"}), 200
    except Exception as e:
        return jsonify({"message": str(e)}), 400

# REPORTS

```



```

@app.route('/reports/courses', methods=['GET'])
@role_required('admin')
def get_courses_report():
    try:
        with get_db_connection() as cnx:
            with cnx.cursor() as cursor:
                cursor.execute("SELECT * FROM vw_courses_50_plus")
                rows = cursor.fetchall()
                courses = [{"courseID": r[0], "ccode": r[1], "cname": r[2], "student_count": r[3]} for r in rows]
            return jsonify(courses), 200
    except Exception as e:
        return jsonify({"message": str(e)}), 400

```

#GET STUDENTS REPORTS

```

@app.route('/reports/students', methods=['GET'])
@role_required('admin')
def get_students_report():
    try:
        with get_db_connection() as cnx:
            with cnx.cursor() as cursor:
                cursor.execute("SELECT * FROM vw_students_5_plus_courses")
                rows = cursor.fetchall()
                students = [{"userID": r[0], "username": r[1], "fname": r[2], "lname": r[3], "course_count": r[4]} for r in
rows]
            return jsonify(students), 200
    except Exception as e:
        return jsonify({"message": str(e)}), 400

```

#GET LECTURERS REPORTS

@app.route('/reports/lecturers', methods=['GET'])

@role_required('admin')

def get_lecturers_report():

try:

with get_db_connection() **as** cnx:

with cnx.cursor() **as** cursor:

cursor.execute("SELECT * FROM vw_lecturers_3_plus_courses")

rows = cursor.fetchall()

lecturers = [{"userID": r[0], "username": r[1], "fname": r[2], "lname": r[3], "course_count": r[4]} **for** r **in** rows]

return jsonify(lecturers), 200

except Exception **as** e:

return jsonify({"message": str(e)}), 400

#GET TOP ENROLLED

@app.route('/reports/top-10-enrolled', methods=['GET'])

def get_top_enrolled():

try:

with get_db_connection() **as** cnx:

with cnx.cursor() **as** cursor:

cursor.execute("SELECT * FROM vw_top_10_enrolled_courses")

rows = cursor.fetchall()

courses = [{"courseID": r[0], "ccode": r[1], "cname": r[2], "student_count": r[3]} **for** r **in** rows]

return jsonify(courses), 200

```

except Exception as e:

    return jsonify({"message": str(e)}), 400


#GET TOP STUDENTS

@app.route('/reports/top-10-students', methods=['GET'])
@role_required('lecturer', 'admin')

def get_top_students():

    try:

        with get_db_connection() as cnx:

            with cnx.cursor() as cursor:

                cursor.execute("SELECT * FROM vw_top_10_students_avg")

                rows = cursor.fetchall()

                students = [{"userID": r[0], "username": r[1], "fname": r[2], "lname": r[3], "overall_average": r[4]} for r in
rows]

            return jsonify(students), 200

    except Exception as e:

        return jsonify({"message": str(e)}), 400


if __name__ == '__main__':

    app.run(debug=True)

```

Bonus Features

Frontend

JWT Authentication

JWT authentication is fully integrated via flask-jwt-extended. Upon login, a signed token is returned containing the user's ID and role. Protected routes use `@jwt_required()` and custom role decorators (`role_required()`) to enforce access control. Tokens should be passed in the Authorization header as `Bearer <token>`.

Indexes for Optimization

The following indexes are created to optimize common queries:

- `idx_enrollments_student` — speeds up lookups of a student's courses
- `idx_enrollments_course` — speeds up member retrieval for a course
- `idx_submissions_student` — speeds up grade/average calculations
- `idx_calendar_date` — speeds up date-filtered event queries
- `idx_threads_forum` — speeds up thread listing per forum
- `idx_replies_thread` / `idx_replies_parent` — speeds up nested reply fetching